

Diário de desenvolvimento — grupo 10

Integrantes e responsabilidades

Integrante	Responsabilidade principal	Classes e entregáveis
Pedro de Mello Porto Daou	Integração, coordenação do paralelismo, comparação e entrega	<code>ParallelMergeSort</code> , <code>Comparador</code> , <code>Benchmark</code> , <code>pom.xml</code> e scripts de empacotamento.
Enzo Garofalo Pampana	Algoritmos de ordenação e revisão da corretude	<code>MergeSort</code> , <code>ProgramaSequencial</code> , <code>MergeSortTest</code> e revisão das tarefas de merge.
João Gabriel da Silva Leite	Entrada/saída, validação e documentação de testes	<code>Teclado</code> , <code>ConsoleVetores</code> , <code>TecladoTest</code> e evidências de execução.

As responsabilidades principais incluíram revisão conjunta de todo o código. As classes `ProgramaParalelo`, `MaiorVetorAproximado` e a documentação final passaram por revisão de todos os três integrantes.

Sessão inicial — 01/10/2026 (Desenvolvimento e Integração)

Horário	Integrante(s)	Atividade
08:00–08:20	Todos	Leitura minuciosa do enunciado e divisão das tarefas: byte, Merge Sort recursivo, <code>processadores - 1</code> ordenadoras, <code>join()</code> , logs e comparação.
08:20–09:00	Pedro de Mello Porto Daou	Organização da estrutura Maven e implementação do <code>ParallelMergeSort</code> : particionamento em N partes, <code>start()</code> , <code>join()</code> e gerenciamento de rodadas de juntadoras.
08:20–09:00	Enzo Garofalo Pampana	Implementação do <code>MergeSort.ordenarIntervalo()</code> e <code>intercalar()</code> ; verificação de índices, tratamento de números negativos e elementos repetidos.

Horário	Integrante(s)	Atividade
08:20–09:00	João Gabriel da Silva Leite	Desenvolvimento da classe <code>ConsoleVetores</code> , gerador de números aleatórios e lógica de exibição completa/parcial; validação dos limites de <code>byte</code> (-128 a 127).
09:00–09:30	Pedro de Mello Porto Daou	Integração das aplicações <code>ProgramaParalelo</code> e <code>Comparador</code> ; implementação da medição precisa de tempo (<code>System.nanoTime()</code>) e cópias isoladas do vetor.
09:00–09:30	Enzo Garofalo Pampana	Implementação de <code>ProgramaSequencial</code> e criação de testes automatizados unitários em <code>MergeSortTest</code> , comparando os resultados com <code>Arrays.sort</code> .
09:00–09:30	João Gabriel da Silva Leite	Implementação da classe <code>Teclado</code> para centralizar a leitura e validação robusta de entradas do usuário via <code>Scanner</code> .
09:30–10:00	Todos	Revisão técnica conjunta do código, testes automatizados e estruturação da documentação.

Fechamento — 04/10/2026 (Validação, Benchmarks e Evidências)

Horário	Integrante(s)	Atividade
16:45–16:47	Pedro de Mello Porto Daou	Revisão do mecanismo de cancelamento cooperativo no <code>ParallelMergeSort</code> e garantia de sincronização total via <code>join()</code> antes de propagar interrupções.
16:45–16:47	Enzo Garofalo Pampana	Revisão das checagens de interrupção e cenários de partição ímpar no Merge Sort.
16:45–16:47	João Gabriel da Silva Leite	Ajuste e validação das mensagens de erro de memória insuficiente e checagem de processadores.
16:47–16:49	Pedro de Mello Porto Daou	Execução e validação da classe <code>Benchmark</code> (aquecimento de JVM, alternância de ordem de execução e cálculo de mediana).
16:47–16:49	Enzo Garofalo Pampana	Validação cruzada com <code>Arrays.sort</code> para vetores massivos de até 10 milhões de elementos.

Horário	Integrante(s)	Atividade
16:47–16:49	João Gabriel da Silva Leite	Execução dos testes interativos no terminal local, extração dos logs e screenshots da execução dos programas.
16:49–16:59	Todos	Validação final da suíte de testes, geração do pacote de entrega e elaboração do relato e conclusão técnica.

Impressões individuais

Pedro de Mello Porto Daou

O ponto mais marcante do trabalho foi coordenar as etapas de sincronização entre as threads. A cada rodada de merge, é fundamental garantir que todas as threads intermediárias tenham finalizado (`join()`) antes que a próxima rodada reduza as partes pela metade. Além disso, medir o tempo com precisão exigiu isolar a criação de objetos e a I/O, focando exclusivamente no algoritmo de ordenação.

Enzo Garofalo Pampana

A implementação do algoritmo Merge Sort em Java exigiu atenção nos detalhes da recursão e no cálculo correto dos índices de início, meio e fim. Garantir que vetores com números negativos, elementos duplicados e partições ímpares funcionassem perfeitamente foi o maior desafio. Usar testes automatizados comparando o resultado com `Arrays.sort` deu total certeza da correteza da solução.

João Gabriel da Silva Leite

Tratar a entrada e saída do usuário exigiu uma validação rigorosa de tipos, limites de `byte` (-128 a 127) e tratamento adequado de exceções. Desenvolver a interface via console de forma limpa e permitir que o usuário imprima apenas um intervalo do vetor final foi muito útil para testar vetores grandes de milhões de elementos sem poluir a tela.

Cronologia verificável do Git e Registros

Data e horário	Registro	Atividade
01/10/2026 09:00:31	Commit inicial <code>d8a7016</code>	Criação do repositório remoto.
01/10/2026 09:01:46	Commit <code>28a49a7</code>	Implementação dos algoritmos sequencial/paralelo, aplicações de console, comparador e testes unitários.
01/10/2026 09:06:11	Commit <code>f67924d</code>	Refatoração da leitura de console para a classe <code>Teclado</code> com validação de tipos.
04/10/2026 16:47:00	Validação de Build	Ajustes finos de cancelamento, exceções de memória e benchmarking.
04/10/2026 16:49:00	Coleta de Evidências	Coleta de logs e resultados para relatórios em <code>docs/evidencias/</code> .
04/10/2026 16:59:20	Commit <code>22eabbe</code>	Finalização de benchmark, relatórios de testes, roteiro de apresentação e scripts.
04/10/2026 19:20:00	Validação Local de Tela	Execução real no ambiente de desenvolvimento com captura de screenshots.

Conclusão técnica

O trabalho atingiu plenamente todos os objetivos propostos. A implementação paralela do Merge Sort provou ser significativamente mais rápida que a versão sequencial para vetores grandes (obtemos speedups expressivos à medida que a massa de dados cresce, superando 5x em vetores de milhões de elementos). A sincronização com `join()` garantiu determinismo e correção total sobre a ordenação dos dados.